



**UNIVERSIDADE FEDERAL DE RORAIMA**



ALONSO FERNANDES CAVALCANTE

RAUL SILVA PAIVA

RUAN FELIPE LAURIANO RAMOS

## **TRABALHO PRÁTICO 2**

**Ordenação e Busca**

Boa Vista, RR

2025

Alonso Fernandes Cavalcante, Raul Silva Paiva, Ruan Felipe Lauriano Ramos

## 1. SELECTION SORT

### 1.1. Definição:

Algoritmo de comparação que divide a lista em duas partes: ordenada e não ordenada. Repetidamente busca o menor elemento da parte não ordenada e o troca com o primeiro elemento dessa parte.

### 1.2. Exemplo Prático:

Organizar livros em uma estante. Percorre-se toda a estante procurando o livro com o título "A...", retira-o e coloca na primeira posição. Em seguida, ignora o primeiro (já ordenado) e percorre o restante buscando o próximo menor título ("B...") para a segunda posição. O processo se repete até o fim.

### 1.3 Vantagens:

- Simplicidade de implementação.
- Realiza o número mínimo de trocas (swaps) entre os algoritmos  $O(n^2)$  básicos.
- In-place (baixa sobrecarga de memória).

### 1.4 Desvantagens:

- Complexidade de tempo  $O(n^2)$  em todos os casos (melhor, médio, pior).
- Geralmente instável (não preserva a ordem de elementos iguais).

## 2. INSERTION SORT

### 2.1. Definição:

Constrói o vetor ordenado um item por vez. Pega um elemento do conjunto não ordenado e o insere na posição correta dentro da parte já ordenada.

### 2.2. Exemplo Prático:

Ordenar cartas de baralho na mão. Segura-se as cartas desordenadas na mão esquerda. Retira-se uma carta por vez com a mão direita e a insere na posição

correta entre as já ordenadas na mão esquerda, deslocando as outras para abrir espaço.

### **2.3. Vantagens:**

- Eficiente para conjuntos de dados pequenos.
- Algoritmo estável.
- Adaptativo: complexidade  $O(n)$  se a lista já estiver quase ordenada.

### **2.4. Desvantagens:**

- Complexidade quadrática  $O(n^2)$  no pior caso e caso médio.
- Ineficiente para grandes volumes de dados.

## **3. BUBBLE SORT**

### **3.1. Definição:**

Percorre repetidamente a lista, comparando elementos adjacentes e trocando-os se estiverem na ordem errada. O processo se repete até que nenhuma troca seja necessária.

### **3.2. Exemplo Prático:**

Fila de estudantes por altura. O professor percorre a fila pedindo para cada par de vizinhos comparar alturas. Se o da esquerda for mais alto, trocam de lugar. Passa-se pela fila várias vezes até que ninguém precise trocar. Os mais altos "flutuam" para o final como bolhas.

### **3.3. Vantagens:**

- Fácil compreensão e implementação.
- Detecta se a lista já está ordenada (com uma flag de otimização).

### **3.4 Desvantagens:**

- Altamente ineficiente ( $O(n^2)$ ).
- Geralmente o mais lento dos algoritmos de ordenação básicos.

## 4. RADIX SORT

### 4.1. Definição:

Algoritmo não comparativo que ordena inteiros processando dígitos individuais. Pode processar do dígito menos significativo para o mais significativo (LSD) ou vice-versa (MSD). Requer um algoritmo de ordenação estável auxiliar.

### 4.2. Exemplo Prático:

Ordenação de documentos por data (Ano, Mês, Dia). Primeiro, separa-se tudo em pilhas baseadas apenas no dia e recolhe. Em seguida, reordena-se com base apenas no mês. Por fim, reordena-se pelo ano. Ao final, os documentos estarão cronologicamente corretos sem comparar as datas completas diretamente.

### 4.3. Vantagens:

- Complexidade linear  $O(nk)$  onde  $k$  é o número de dígitos.
- Mais rápido que algoritmos baseados em comparação ( $O(n \log n)$ ) para inteiros grandes.

### 4.4. Desvantagens:

- Restrito a tipos de dados específicos (inteiros, strings).
- Consome mais memória devido ao uso de arrays auxiliares.

## 5. QUICK SORT

### 5.1. Definição:

Algoritmo de divisão e conquista. Seleciona um elemento como "pivô" e particiona o array ao redor dele (menores à esquerda, maiores à direita). As sub-listas são ordenadas recursivamente.

### 5.2. Exemplo Prático:

Arrumar sala por idade. Escolhe-se um aluno aleatório (pivô) e ordena: "Mais novos para a esquerda, mais velhos para a direita". A sala divide-se em dois grupos.

Repete-se o processo dentro de cada grupo recursivamente até formar uma fila indiana.

### 5.3. Vantagens:

- Muito rápido na prática; complexidade média  $O(n \log n)$ .
- Trabalha in-place (pouca memória adicional).

### 5.4. Desvantagens:

- Pior caso é  $O(n^2)$  (se o pivô for mal escolhido).
- Instável.

## 6. MERGE SORT

### 6.1. Definição:

Algoritmo de divisão e conquista que divide o array pela metade recursivamente até obter sub-arrays unitários, e então os mescla (merge) de forma ordenada.

### 6.2. Exemplo Prático:

Mesclar pilhas de provas. Imagine duas turmas (A e B) com provas já em ordem alfabética. Para criar uma pilha única, compara-se apenas o topo da pilha A com o topo da pilha B. A prova que vier primeiro no alfabeto vai para a pilha final. Repete-se comparando sempre os topos atuais.

### 6.3. Vantagens:

- Garantia de complexidade  $O(n \log n)$  no pior caso.
- Estável.

### 6.4. Desvantagens:

- Requer espaço de memória auxiliar  $O(n)$ .
- Mais lento que Quick Sort para arrays pequenos devido à sobrecarga de chamadas.

## **7. BUSCA SEQUENCIAL (LINEAR)**

### **7.1. Definição:**

Método de busca onde cada elemento do vetor é verificado sequencialmente até que o valor desejado seja encontrado ou o final do vetor seja alcançado.

### **7.2. Exemplo Prático:**

Procurar camisa no cesto de roupa suja. As roupas estão misturadas. Pega-se peça por peça, verifica-se se é a camisa desejada e, se não for, passa para a próxima. No pior caso, verifica-se o cesto inteiro.

### **7.3. Vantagens:**

- Não requer que os dados estejam ordenados.
- Implementação trivial.

### **7.4. Desvantagens:**

- Lento para grandes conjuntos de dados: complexidade  $O(n)$ .

## **8. BUSCA BINÁRIA**

### **8.1. Definição:**

Algoritmo eficiente que funciona dividindo repetidamente o intervalo de busca pela metade. Requer dados previamente ordenados.

### **8.2. Exemplo Prático:**

Procurar palavra no dicionário. Abre-se o dicionário no meio. Se a palavra buscada vem antes da página aberta, ignora-se a metade final e foca-se na primeira metade. Abre o meio dessa nova seção e repete a lógica, reduzindo a busca pela metade a cada passo.

### **8.3. Vantagens:**

- Alta eficiência: complexidade  $O(\log n)$ .

- Ideal para grandes volumes de dados estáticos.

#### **8.4. Desvantagens:**

- Exige que o vetor esteja ordenado.
- Acesso aleatório aos dados é necessário.

### **9. HEAP SORT**

#### **9.1. Definição:**

Baseado na estrutura de dados Heap (árvore binária completa). O algoritmo constrói um Max-Heap e repetidamente extrai o maior elemento (raiz), colocando-o no final do array.

#### **9.2. Exemplo Prático:**

Triagem em pronto-socorro. Pacientes chegam aleatoriamente, mas o sistema organiza a fila para que o mais grave esteja sempre no topo (raiz). O médico atende o mais grave, retira-o, e o sistema reorganiza a estrutura para que o próximo mais grave assuma o topo imediatamente.

#### **9.3. Vantagens:**

- Complexidade  $O(n \log n)$  consistente.
- In-place (não requer memória auxiliar significativa).

#### **9.4. Desvantagens:**

- Instável.
- Geralmente mais lento que o Quick Sort devido à localidade de referência (cache).

## REFERÊNCIAS BIBLIOGRÁFICAS

1. **CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C.** *Algoritmos: Teoria e Prática*. 3. ed. Rio de Janeiro: Elsevier, 2012.
2. **KNUTH, D. E.** *The Art of Computer Programming, Volume 3: Sorting and Searching*. 2. ed. Reading: Addison-Wesley, 1998.
3. **SEEDGEWICK, R.** *Algorithms in C, Parts 1-4: Fundamentals, Data Structures, Sorting, Searching*. 3. ed. Reading: Addison-Wesley, 1997.
4. **TENENBAUM, A. M.; LANGSAM, Y.; AUGENSTEIN, M. J.** *Estruturas de Dados Usando C*. 1. ed. São Paulo: Pearson Makron Books, 1995.